

# **Test Report**

## **(Task 3)**

**Name:**Akhilesh.M

**CIN ID:**FIT/JAN26/5527

**Github:** <https://github.com/ig-immortal>

# **API Security Risk Analysis**

**Modern Application Programming Interfaces (APIs) are the driving force behind today's digital transformation. They enable mobile applications, SaaS platforms, and interactive dashboards to communicate seamlessly, exchange data, and deliver rich user experiences. APIs allow businesses to integrate services, scale rapidly, and innovate faster by connecting diverse systems in real time. From enabling payment gateways in mobile apps to powering analytics dashboards in enterprise SaaS, APIs are the invisible infrastructure that keeps the digital economy running smoothly.**

**Yet, this immense power comes with significant risks. APIs often act as direct entry points into critical systems and sensitive data. If they are not properly secured, attackers can exploit vulnerabilities with devastating consequences. Common threats include:**

- Sensitive Data Exposure: Insecure endpoints can leak personal, financial, or proprietary information.**
- Authentication Bypass: Weak or misconfigured authentication mechanisms may allow unauthorized access.**
- Endpoint Abuse: Malicious actors can manipulate APIs to perform unintended actions, such as privilege escalation or fraudulent transactions.**
- System Overload: Attackers may flood APIs with excessive requests, leading to denial-of-service attacks and system downtime.**

**These risks highlight why API security must be treated as a top priority. Organizations should implement strong authentication and authorization protocols, encrypt data in transit, and enforce rate limiting to prevent abuse. Continuous monitoring and logging are essential to detect suspicious activity early. Regular penetration testing and adherence to secure coding practices further strengthen defenses.**

**Moreover, businesses should adopt a “security by design” approach, embedding safeguards into APIs from the outset rather than treating them as afterthoughts. With APIs powering critical services across mobile apps, SaaS platforms, and dashboards, their protection is not optional—it is fundamental to maintaining trust, reliability, and resilience in the digital age.**

**In conclusion, modern APIs are both a powerful enabler and a potential vulnerability. They unlock innovation and connectivity, but without robust**

security, they can expose organizations to severe risks. Securing APIs ensures that businesses can continue to harness their full potential while safeguarding users and systems against evolving threats.

## **OWASP API Top 10**

The OWASP API Top 10 highlights the most critical security risks facing APIs today, including broken authorization, weak authentication, misconfiguration, and unsafe consumption of third-party APIs. Addressing these risks is essential to protect sensitive data and ensure reliable system performance.

---

### **📌 OWASP API Top 10 (2023) – Summary**

APIs power mobile apps, SaaS platforms, and dashboards, but insecure APIs expose organizations to serious threats. The OWASP API Security Top 10 (2023) identifies the most common vulnerabilities:

1. **Broken Object Level Authorization** – Attackers exploit weak access controls to retrieve or manipulate data objects.
2. **Broken Authentication** – Flaws in login or token handling allow impersonation.
3. **Broken Object Property Level Authorization** – Excessive data exposure or mass assignment due to poor property validation.
4. **Unrestricted Resource Consumption** – APIs can be overloaded by excessive requests, leading to denial of service.
5. **Broken Function Level Authorization** – Attackers invoke privileged functions without proper checks.
6. **Unrestricted Access to Sensitive Business Flows** – Critical workflows (e.g., payments) can be abused if not protected.
7. **Server-Side Request Forgery (SSRF)** – APIs may be tricked into making unauthorized internal/external requests.
8. **Security Misconfiguration** – Default settings, verbose error messages, or missing patches expose systems.
9. **Improper Inventory Management** – Shadow or outdated APIs increase attack surface.
10. **Unsafe Consumption of APIs** – Trusting external APIs without validation introduces risks.

**APIs are powerful but vulnerable. Organizations must enforce strong authentication, authorization checks, rate limiting, encryption, monitoring, and secure design practices to mitigate these risks.**

## **API Layer APIs**

|                             |                                                                                                                |
|-----------------------------|----------------------------------------------------------------------------------------------------------------|
| <b><u>IPstack</u></b>       | <b>Locate and Identify Website Visitors by IP Address</b>                                                      |
| <b><u>Marketstack</u></b>   | <b>Free, easy-to-use REST API interface delivering worldwide stock market data in JSON format</b>              |
| <b><u>Weatherstack</u></b>  | <b>Retrieve instant, accurate weather information for any location in the world in lightweight JSON format</b> |
| <b><u>Numverify</u></b>     | <b>Global Phone Number Validation &amp; Lookup JSON API</b>                                                    |
| <b><u>Fixer</u></b>         | <b>Fixer is a simple and lightweight API for current and historical foreign exchange (forex) rates.</b>        |
| <b><u>Aviationstack</u></b> | <b>Free, real-time flight status and global Aviation data API</b>                                              |

## **Public API(demo)**

| <b>API</b>                             | <b>Description</b>                                          |
|----------------------------------------|-------------------------------------------------------------|
| <b><u>Phone Validation</u></b>         | <b>Validate phone numbers globally</b>                      |
| <b><u>apilayer numverify</u></b>       | <b>Phone number validation</b>                              |
| <b><u>Cloudmersive Validate</u></b>    | <b>Validate international phone numbers</b>                 |
| <b><u>Phone Specification</u></b>      | <b>Rest Api for Phone specifications</b>                    |
| <b><u>Veriphone</u></b>                | <b>Phone number validation &amp; carrier lookup</b>         |
| <b><u>Creative Commons Catalog</u></b> | <b>Search among openly licensed and public domain works</b> |
| <b><u>Datamuse</u></b>                 | <b>Word-finding query engine</b>                            |
| <b><u>Drupal.org</u></b>               | <b>Drupal.org</b>                                           |
| <b><u>Evil Insult Generator</u></b>    | <b>Evil Insults</b>                                         |

## GitHub Contribution Chart Generator

Create an image of your GitHub contributions

## GitHub ReadMe Stats

Add dynamically generated statistics to your GitHub profile ReadMe

## Metabase

An open source Business Intelligence server to share data and analytics inside your company

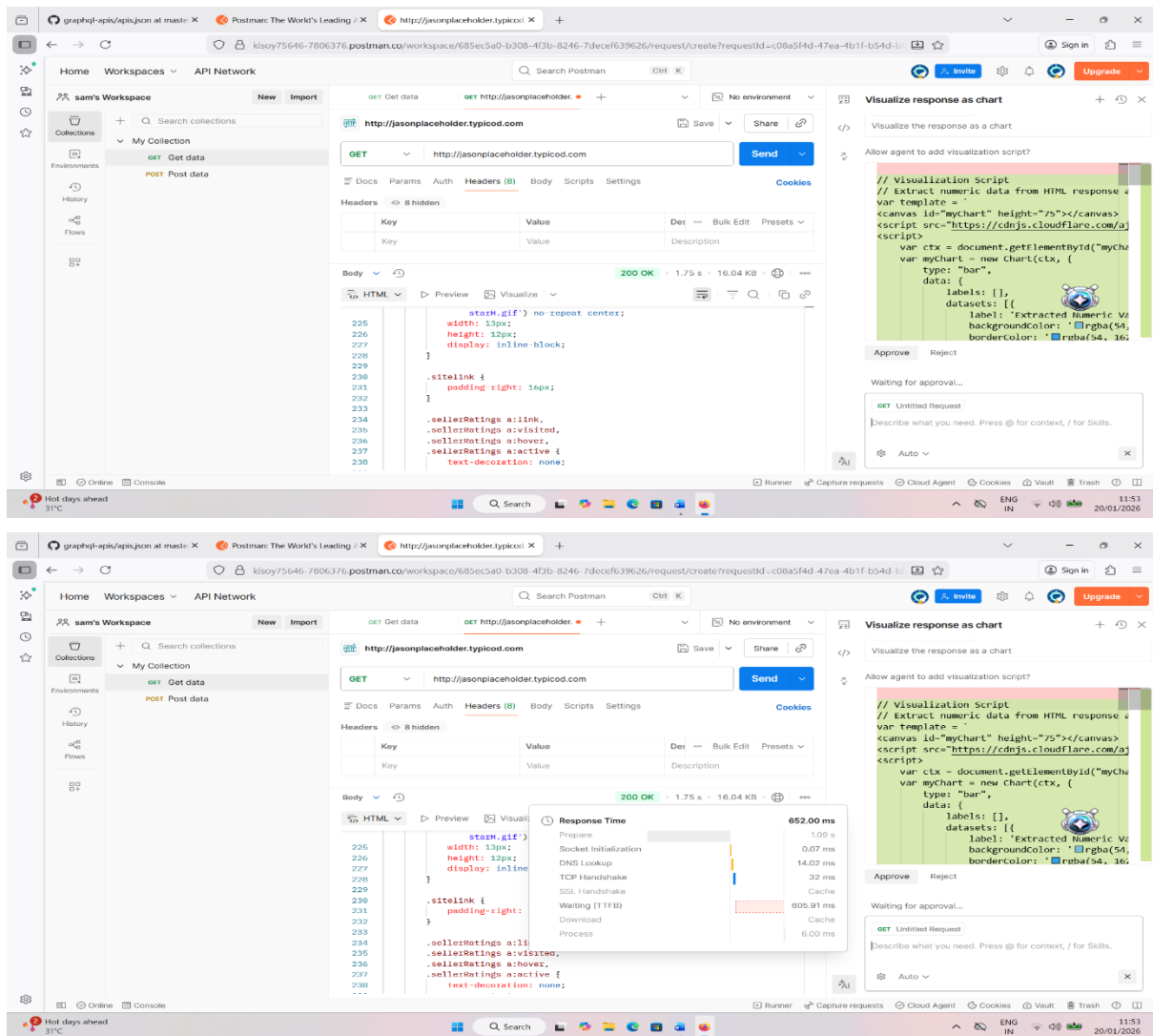
## Shields

Concise, consistent, and legible badges in SVG and raster format

# API Testing

Demo api : <http://jsonplaceholder.typicode.com>

Tool : [www.postman.com](http://www.postman.com)



Postman interface showing a GET request to `http://jsonplaceholder.typicode.com`. The response is a 200 OK status with a 1.18s duration and 16.04 KB size. The response body is HTML content. A visualization script is shown on the right, which extracts numeric data from the HTML response and creates a chart.

```
// Visualization Script
// Extract numeric data from HTML response
var template =
<canvas id="mychart" height="75"></canvas>
<script src="https://cdn.jsdelivr.net/npm/chart.js">
</script>
var ctx = document.getElementById("myChart")
var myChart = new Chart(ctx, {
  type: "bar",
  data: {
    labels: [],
    datasets: [
      {
        label: "Extracted Numeric Value",
        backgroundColor: "rgb(54, 54, 54)",
        borderColor: "rgb(54, 54, 54)",

```

Postman interface showing a GET request to `http://jsonplaceholder.typicode.com`. The response is a 200 OK status with a 1.18s duration and 16.04 KB size. The response body is HTML content. A visualization script is shown on the right, which extracts numeric data from the HTML response and creates a chart. A response time breakdown is visible, showing a total of 401.00 ms.

| Step                  | Duration  |
|-----------------------|-----------|
| Prepare               | 777.00 ms |
| Socket Initialization | 0.07 ms   |
| DNS Lookup            | 13.77 ms  |
| TCP Handshake         | 30 ms     |
| SSL Handshake         | Cache     |
| Waiting (TTFB)        | 358.58 ms |
| Download              | 1.00 ms   |
| Process               | 2.00 ms   |

Postman interface showing a GET request to `http://regres.in`. The response is a 200 OK status with a 1.14s duration and 285.52 KB size. The response body is HTML content. A visualization script is shown on the right, which extracts numeric data from the HTML response and creates a chart.

```
// Visualization Script
// Extract numeric data from HTML response
var template =
<canvas id="mychart" height="75"></canvas>
<script src="https://cdn.jsdelivr.net/npm/chart.js">
</script>
var ctx = document.getElementById("myChart")
var myChart = new Chart(ctx, {
  type: "bar",
  data: {
    labels: [],
    datasets: [
      {
        label: "Extracted Numeric Value",
        backgroundColor: "rgb(54, 54, 54)",
        borderColor: "rgb(54, 54, 54)",

```

Postman interface showing a GET request to `http://regres.in`. The response is a 200 OK status with a 1.14s duration and 285.52 KB size. The response body is HTML content. A visualization script is shown on the right, which extracts numeric data from the HTML response and creates a chart. A response time breakdown is visible, showing a total of 805.00 ms.

| Step                  | Duration  |
|-----------------------|-----------|
| Prepare               | 338.00 ms |
| Socket Initialization | 0.08 ms   |
| DNS Lookup            | 44.08 ms  |
| TCP Handshake         | 5 ms      |
| SSL Handshake         | 6.98 ms   |
| Waiting (TTFB)        | 408.87 ms |
| Download              | 340.00 ms |
| Process               | 1.00 ms   |

# **API Security Checklist**

**APIs are the backbone of modern applications, enabling seamless communication between systems, services, and users. However, their openness and accessibility make them prime targets for cyberattacks. An API Security Checklist serves as a structured guide to ensure that every stage of API development, deployment, and maintenance incorporates robust security practices. It helps developers, architects, and security teams identify vulnerabilities, enforce authentication, protect sensitive data, and maintain compliance with industry standards. By following a checklist, organizations can systematically address common risks such as broken authentication, excessive data exposure, and injection attacks. This proactive approach not only safeguards business assets but also builds user trust and resilience against evolving threats. In essence, an API Security Checklist is more than a technical tool—it's a strategic framework for creating secure, reliable, and future-ready APIs that support innovation without compromising safety.**

## **API Security Checklist**

**Checklist of the most important security countermeasures when designing, testing, and releasing your API.**

---

### **Authentication**

- **Don't use Basic Auth. Use standard authentication instead (e.g., [JWT](#)).**
- **Don't reinvent the wheel in Authentication, token generation, password storage. Use the standards.**
- **Use Max Retry and jail features in Login.**
- **Use encryption on all sensitive data.**

### **JWT (JSON Web Token)**

- **Use a random complicated key (JWT Secret) to make brute forcing the token very hard.**
- **Don't extract the algorithm from the header. Force the algorithm in the backend (HS256 or RS256).**
- **Make token expiration (TTL, RTTL) as short as possible.**
- **Don't store sensitive data in the JWT payload, it can be decoded [easily](#).**

- **Avoid storing too much data. JWT is usually shared in headers and they have a size limit.**

### **Access**

- **Limit requests (Throttling) to avoid DDoS / brute-force attacks.**
- **Use HTTPS on server side with TLS 1.2+ and secure ciphers to avoid MITM (Man in the Middle Attack) and ensure Host header matches the SNI.**
- **Use HSTS header with SSL to avoid SSL Strip attacks.**
- **Turn off directory listings.**
- **For private APIs, allow access only from safelisted IPs/hosts.**

### **Authorization**

#### **OAuth**

- **Always validate redirect\_uri server-side to allow only safelisted URLs.**
- **Always try to exchange for code and not tokens (don't allow response\_type=token).**
- **Use state parameter with a random hash to prevent CSRF on the OAuth authorization process.**
- **Define the default scope, and validate scope parameters for each application.**

### **Input**

- **Use the proper HTTP method according to the operation: GET (read), POST (create), PUT/PATCH (replace/update), and DELETE (to delete a record), and respond with 405 Method Not Allowed if the requested method isn't appropriate for the requested resource.**
- **Validate content-type on request Accept header (Content Negotiation) to allow only your supported format (e.g., application/xml, application/json, etc.) and respond with 406 Not Acceptable response if not matched.**
- **Validate content-type of posted data as you accept (e.g., application/x-www-form-urlencoded, multipart/form-data, application/json, etc.).**
- **Validate user input to avoid common vulnerabilities (e.g., XSS, SQL-Injection, Remote Code Execution, etc.).**



- **Don't use any sensitive data (credentials, Passwords, security tokens, or API keys) in the URL, but use standard Authorization header.**
- **Use only server-side encryption.**
- **Use an API Gateway service to enable caching, Rate Limit policies (e.g., Quota, Spike Arrest, or Concurrent Rate Limit) and deploy APIs resources dynamically.**

### **Processing**

- **Check if all the endpoints are protected behind authentication to avoid broken authentication process.**
- **User own resource ID should be avoided. Use /me/orders instead of /user/654321/orders.**
- **Don't auto-increment IDs. Use UUID instead.**
- **If you are parsing XML data, make sure entity parsing is not enabled to avoid XXE (XML external entity attack).**
- **If you are parsing XML, YAML or any other language with anchors and refs, make sure entity expansion is not enabled to avoid Billion Laughs/XML bomb via exponential entity expansion attack.**
- **Use a CDN for file uploads.**
- **If you are dealing with huge amount of data, use Workers and Queues to process as much as possible in background and return response fast to avoid HTTP Blocking.**
- **Do not forget to turn the DEBUG mode OFF.**
- **Use non-executable stacks when available.**

### **Output**

- **Send X-Content-Type-Options: nosniff header.**
- **Send X-Frame-Options: deny header.**
- **Send Content-Security-Policy: default-src 'none' header.**
- **Remove fingerprinting headers - X-Powered-By, Server, X-AspNet-Version, etc.**
- **Force content-type for your response. If you return application/json, then your content-type response is application/json.**
- **Don't return sensitive data like credentials, passwords, or security tokens.**

- **Return the proper status code according to the operation completed. (e.g., 200 OK, 400 Bad Request, 401 Unauthorized, 405 Method Not Allowed, etc.).**

### **CI & CD**

- **Audit your design and implementation with unit/integration tests coverage.**
- **Use a code review process and disregard self-approval.**
- **Ensure that all components of your services are statically scanned by AV software before pushing to production, including vendor libraries and other dependencies.**
- **Continuously run security tests (static/dynamic analysis) on your code.**
- **Check your dependencies (both software and OS) for known vulnerabilities.**
- **Design a rollback solution for deployments.**

### **Monitoring**

- **Use centralized logins for all services and components.**
- **Use agents to monitor all traffic, errors, requests, and responses.**
- **Use alerts for SMS, Slack, Email, Telegram, Kibana, Cloudwatch, etc.**
- **Ensure that you aren't logging any sensitive data like credit cards, passwords, PINs, etc.**
- **Use an IDS and/or IPS system to monitor your API requests and instances.**

## **API Security Risk**

**APIs face major security risks such as broken authentication, excessive data exposure, and misconfigurations, making them prime targets for attackers. The OWASP API Security Top 10 highlights these vulnerabilities as critical issues organizations must address to safeguard sensitive data and business operations.**

---

### **Key Security Risks in APIs**

#### **1. Broken Authentication**

- **Weak or improperly implemented authentication allows attackers to impersonate users.**
- **Risks include account takeover and unauthorized access to sensitive data.**

## **2. Excessive Data Exposure**

- **APIs often return more data than necessary, exposing sensitive information.**
- **Attackers can exploit verbose responses to harvest personal or business data.**

## **3. Broken Object Level Authorization**

- **Failure to enforce proper access controls lets attackers manipulate object IDs to access unauthorized resources.**

## **4. Security Misconfiguration**

- **Default settings, exposed error messages, or incomplete patches create exploitable weaknesses.**
- **Common in APIs deployed quickly without thorough hardening.**

## **5. Injection Attacks**

- **APIs vulnerable to SQL, NoSQL, or command injection can allow attackers to manipulate backend systems.**

## **6. Unrestricted Resource Consumption**

- **Poor rate limiting enables denial-of-service (DoS) attacks by overwhelming the API with requests.**

## **7. Server-Side Request Forgery (SSRF)**

- **Malicious requests can trick APIs into fetching internal or sensitive resources.**

## **8. Improper Inventory Management**

- **Forgotten or outdated API versions remain exposed, creating unmonitored attack surfaces.**

## **9. Unsafe Consumption of APIs**

- **Trusting third-party APIs without validation can introduce vulnerabilities into your system.**

# **Classify Security Risk of API**

| <b>Risk</b>                      | <b>Impact on Business</b>                | <b>Example Exploit</b>                |
|----------------------------------|------------------------------------------|---------------------------------------|
| <b>Broken Authentication</b>     | <b>Account takeover, data theft</b>      | <b>Weak token validation</b>          |
| <b>Excessive Data Exposure</b>   | <b>Leakage of PII, compliance issues</b> | <b>Overly detailed JSON responses</b> |
| <b>Security Misconfiguration</b> | <b>System compromise</b>                 | <b>Default admin credentials</b>      |
| <b>Injection Attacks</b>         | <b>Database corruption, data loss</b>    | <b>SQL injection via API input</b>    |
| <b>Unrestricted Resource Use</b> | <b>Service downtime</b>                  | <b>Flooding API with requests</b>     |
| <b>SSRF</b>                      | <b>Internal network exposure</b>         | <b>API fetching attacker's URL</b>    |

---

## **⚠ Why This Matters**

- **APIs are the backbone of modern apps—from banking to healthcare—and attackers exploit them because they often expose direct access to data and services.**
- **Regulatory compliance (GDPR, HIPAA) requires strict API security to avoid fines and reputational damage.**
- **Business continuity depends on preventing DoS attacks and unauthorized access.**

## **Remediation For Secure API**

**To secure APIs from attacks, organizations must enforce strong authentication, apply strict authorization, sanitize inputs, encrypt data, and continuously monitor for anomalies. The most effective remediation strategies combine preventive controls (like rate limiting and token validation) with proactive measures (like regular penetration testing and patching).**

---

# **Remediation Strategies for API Security**

## **1. Authentication & Authorization**

- **Use OAuth 2.0 or JWT tokens for secure authentication.**
- **Enforce role-based access control (RBAC) to ensure users only access permitted resources.**
- **Implement multi-factor authentication (MFA) for sensitive APIs.**

## **2. Data Protection**

- **Encrypt data in transit (TLS/HTTPS) and at rest.**
- **Avoid exposing unnecessary fields in API responses (prevent excessive data exposure).**
- **Mask or tokenize sensitive information such as PII and financial data.**

## **3. Input Validation & Threat Prevention**

- **Sanitize all inputs to prevent SQL/NoSQL injection.**
- **Use API gateways with built-in threat detection and schema validation.**
- **Block malicious bots with CAPTCHAs and bot management tools.**

## **4. Rate Limiting & Resource Management**

- **Apply rate limiting and throttling to prevent denial-of-service (DoS) attacks.**
- **Monitor API usage patterns to detect anomalies or abuse.**

## **5. Security Configuration & Version Control**

- **Disable unused endpoints and remove deprecated API versions.**
- **Harden configurations: avoid default credentials, verbose error messages, and open ports.**
- **Maintain an API inventory to track all active endpoints.**

## **6. Monitoring & Logging**

- **Implement centralized logging with alerts for suspicious activity.**
- **Use runtime protection tools to detect and block attacks in real time.**
- **Regularly audit logs for unauthorized access attempts.**

## **7. Testing & Continuous Improvement**

- **Conduct penetration testing and fuzz testing for APIs.**
- **Automate security scans in CI/CD pipelines.**

- **Patch vulnerabilities promptly and update dependencies.**

## **Quick Comparison of Remediation Approaches**

| <b>Risk</b>                      | <b>Remediation Strategy</b>                  | <b>Example Tool/Method</b> |
|----------------------------------|----------------------------------------------|----------------------------|
| <b>Broken Authentication</b>     | <b>OAuth 2.0, JWT, MFA</b>                   | <b>Auth0, Okta</b>         |
| <b>Excessive Data Exposure</b>   | <b>Response filtering, data masking</b>      | <b>API Gateway filters</b> |
| <b>Injection Attacks</b>         | <b>Input sanitization, schema validation</b> | <b>OWASP ESAPI</b>         |
| <b>DoS / Resource Abuse</b>      | <b>Rate limiting, throttling</b>             | <b>NGINX, Kong</b>         |
| <b>Security Misconfiguration</b> | <b>Harden configs, patching</b>              | <b>CIS Benchmarks</b>      |
| <b>Shadow APIs</b>               | <b>Inventory management, monitoring</b>      | <b>API discovery tools</b> |

---

### **Risks & Trade-offs**

- **Overly strict rate limiting may block legitimate users.**
- **Encryption overhead can impact performance if not optimized.**
- **Third-party API reliance introduces external risks—always validate responses.**